

8. Funciones

8.1 Implementando funciones para reutilizar código

En programación es muy frecuente reutilizar código, es decir, usar código ya existente. Cuando una parte de un programa requiere una funcionalidad que ya está implementada en otro programa no tiene mucho sentido emplear tiempo y energía en implementarla otra vez.

Una función es un trozo de código que realiza una tarea muy concreta y que se puede incluir en cualquier programa cuando hace falta resolver esa tarea. Opcionalmente, las funciones aceptan una entrada (parámetros de entrada) y devuelven una salida.

Observa el siguiente ejemplo. Se trata de un programa que pide un número por teclado y luego dice si el número introducido es o no es primo.

```
/**
 * Dice si un número es o no es primo (sin funciones)
 *
 * @author Luis José Sánchez
 */
public class NumeroPrimo {
    public static void main(String[] args) {

        System.out.print("Introduce un número entero positivo: ");
        int n = Integer.parseInt(System.console().readLine());

        boolean esPrimo = true;
        for (int i = 2; i < n; i++) {
            if ((n % i) == 0) {
                esPrimo = false;
            }
        }

        if (esPrimo) {
            System.out.println("El " + n + " es primo.");
        } else {
            System.out.println("El " + n + " no es primo.");
        }
    }
}
```

Podemos intuir que la tarea de averiguar si un número es o no primo será algo que utilizaremos con frecuencia más adelante así que podemos aislar el trozo de código que realiza ese cometido para usarlo con comodidad en otros programas.

```
/**
 * Dice si un número es o no es primo usando una función
 *
 * @author Luis José Sánchez
 */
public class NumeroPrimoConFuncion {

    // Programa principal //////////////////////////////////////

    public static void main(String[] args) {

        System.out.print("Introduzca un número entero positivo: ");
        int n = Integer.parseInt(System.console().readLine());

        if (esPrimo(n)) {
            System.out.println("El " + n + " es primo.");
        } else {
            System.out.println("El " + n + " no es primo.");
        }
    }

    // Funciones //////////////////////////////////////

    /**
     * Comprueba si un número entero positivo es primo o no.
     * Un número es primo cuando únicamente es divisible entre
     * él mismo y la unidad.
     *
     * @param x un número entero positivo
     * @return <code>true</code> si el número es primo
     * @return <code>false</code> en caso contrario
     */
    public static boolean esPrimo(int x) {

        for (int i = 2; i < x; i++) {
            if ((x % i) == 0) {
                return false;
            }
        }

        return true;
    }
}
```

```
}
```

Cada función tiene una cabecera y un cuerpo. En el ejemplo anterior la cabecera es

```
public static boolean esPrimo(int x)
```

De momento no vamos a explicar qué significa `public static`, lo dejaremos para cuando veamos el capítulo 9 “Programación orientada a objetos”, por ahora lo escribiremos tal cual cada vez que definamos una función. A continuación se escribe el tipo de dato que devuelve la función, en este caso es `boolean` porque la función devolverá siempre `true` (verdadero) o `false` falso. Lo último que lleva la cabecera son los parámetros encerrados entre paréntesis. Esos parámetros son los valores que se le pasan a la función para que realice los cálculos. En este caso concreto, el parámetro que se pasa es `x`, o sea, el número que queremos saber si es primo o no. Es necesario indicar siempre el tipo de cada parámetro; en esta ocasión, el parámetro que se pasa es de tipo entero (`int`).

8.2 Comentarios de funciones

Los comentarios tienen un papel muy importante en los programas ya que, aunque no se compilan ni se ejecutan, permiten describir, aclarar o dar información relevante sobre qué hace exactamente el código. Un programa bien comentado es mucho más fácil de corregir, mantener y mejorar que un programa mal comentado o que simplemente no tiene comentarios.

Existe una herramienta llamada `javadoc` que crea unos documentos en HTML con información sobre programas escritos en Java. Para ello, `javadoc` utiliza los comentarios que se han hecho y, además, la meta-información que se incluye en esos comentarios. Esta meta-información es muy fácil de distinguir pues va precedida siempre de un carácter `@`.

Te habrás dado cuenta que en los comentarios de los programas que hemos visto en este manual, se indica el nombre del autor mediante `@author`.

Si los comentarios de los programas en general son importantes, debemos prestar especial atención a los comentarios de las funciones ya que posiblemente serán usadas por otros programadores que querrán saber exactamente cómo se utilizan.

Con la etiqueta `@param` seguida de un nombre de parámetro se indica qué parámetro espera como entrada la función. Si una función acepta varios parámetros, se especificarán varias etiquetas `@param` en los comentarios.

Mediante la etiqueta `@return` especificamos qué devuelve exactamente la función. Aunque el tipo de dato que se devuelve ya viene indicado en la cabecera de la función, mediante la etiqueta `@return` podemos explicarlo con más detalle.

En los comentarios se pueden incluir elementos de HTML como `<code>`, ` `, etc. ya que los entiende perfectamente `javadoc`.

Puedes hacer una prueba generando la documentación del ejemplo ejecutando la siguiente línea en una ventana de terminal. Como tenemos tildes en los comentarios, si queremos que salgan bien en las páginas de documentación, debemos indicar la codificación de caracteres.

```
javadoc -encoding UTF-8 -charset UTF-8 -docencoding UTF-8 NumeroPrimoConFuncion.java
```

Para aprender más sobre `javadoc` puedes consultar [la documentación oficial de Oracle¹](http://www.oracle.com/technetwork/java/javase/documentation/index-137868.html).

8.3 Creación de bibliotecas de rutinas mediante paquetes

Si la función `esPrimo()` va a ser usada en tres programas diferentes se puede copiar y pegar su código en cada uno de los programas, pero hay una solución mucho más elegante y práctica.

Las funciones de un determinado tipo (por ejemplo funciones matemáticas) se pueden agrupar para crear un paquete (`package`) que luego se importará desde el programa que necesite esas funciones.

Cada paquete se corresponde con un directorio. Por tanto, si hay un paquete con nombre `matematicas` debe haber un directorio llamado también `matematicas` en la misma ubicación del programa que importa ese paquete (normalmente el programa principal).

Las funciones se pueden agrupar dentro de un paquete de dos maneras diferentes. Puede haber subpaquetes dentro de un paquete; por ejemplo, si quisiéramos dividir las funciones matemáticas en funciones relativas al cálculo de áreas y volúmenes de figuras geométricas y funciones relacionadas con cálculos estadísticos, podríamos crear dos directorios dentro de `matematicas` con nombres `geometria` y `estadistica` respectivamente. Estos subpaquetes se llamarían `matematicas.geometria` y `matematicas.estadistica`. Otra manera de agrupar las funciones dentro de un mismo paquete consiste en crear varios ficheros dentro de un mismo directorio. En este caso se podrían crear los ficheros `Geometria.java` y `Estadistica.java`.

Entenderemos mejor todos estos conceptos con un ejemplo completo. Vamos a crear un paquete con nombre `matematicas` que contenga dos clases: `Varias` (para funciones matemáticas de propósito general) y `Geometria`. Por tanto en el disco duro, tendremos una carpeta con nombre `matematicas` que contiene los ficheros `Varias.java` y `Geometria.java`. El contenido de estos ficheros se muestra a continuación.

¹<http://www.oracle.com/technetwork/java/javase/documentation/index-137868.html>

```
package matematicas;

/**
 * Funciones matemáticas de propósito general
 *
 * @author Luis José Sánchez
 */
public class Varias {

    /**
     * Comprueba si un número entero positivo es primo o no.
     * Un número es primo cuando únicamente es divisible entre
     * él mismo y la unidad.
     *
     * @param x un número entero positivo
     * @return <code>true</code> si el número es primo
     * @return <code>false</code> en caso contrario
     */
    public static boolean esPrimo(int x) {

        for (int i = 2; i < x; i++) {
            if ((x % i) == 0) {
                return false;
            }
        }

        return true;
    }

    /**
     * Devuelve el número de dígitos que contiene un número entero
     *
     * @param x un número entero
     * @return la cantidad de dígitos que contiene el número
     */
    public static int digitos(int x) {

        if (x == 0) {
            return 1;
        } else {
            int n = 0;
            while (x > 0) {
                x = x / 10;
                n++;
            }
            return n;
        }
    }
}
```

```

    }
}
}

```

Se incluye a continuación `Geometria.java`. Recuerda que tanto `Varias.java` como `Geometria.java` se encuentran dentro del directorio `matematicas`.

```

package matematicas;

/**
 * Funciones geométricas
 *
 * @author Luis José Sánchez
 */
public class Geometria {

    /**
     * Calcula el volumen de un cilindro.
     * Tanto el radio como la altura se deben proporcionar en las
     * mismas unidades para que el resultado sea congruente.
     *
     * @param r radio del cilindro
     * @param h altura del cilindro
     * @return volumen del cilindro
     */
    public static double volumenCilindro(double r, double h) {
        return Math.PI * r * r * h;
    }

    /**
     * Calcula la longitud de una circunferencia a partir del radio.
     *
     * @param r radio de la circunferencia
     * @return longitud de la circunferencia
     */
    public static double longitudCircunferencia(double r) {
        return 2 * Math.PI * r;
    }
}

```

Observa que en ambos ficheros se especifica que las clases declaradas (y por tanto las funciones que se definen dentro) pertenecen al paquete `matematicas` mediante la línea `package matematicas`. Ahora ya podemos probar las funciones desde un programa externo. El programa `PruebaFunciones.java` está fuera de la carpeta `matematicas`, justo en un nivel superior en la estructura de directorios.

```
import matematicas.Varias;
import matematicas.Geometria;

/**
 * Prueba varias funciones
 *
 * @author Luis José Sánchez
 */
public class PruebaFunciones {
    public static void main(String[] args) {

        int n;

        // Prueba esPrimo()

        System.out.print("Introduzca un número entero positivo: ");
        n = Integer.parseInt(System.console().readLine());

        if (matematicas.Varias.esPrimo(n)) {
            System.out.println("El " + n + " es primo.");
        } else {
            System.out.println("El " + n + " no es primo.");
        }

        // Prueba digitos()

        System.out.print("Introduzca un número entero positivo: ");
        n = Integer.parseInt(System.console().readLine());

        System.out.println(n + " tiene " + matematicas.Varias.digitos(n) + " dígitos.");

        // Prueba volumenCilindro()

        double r, h;

        System.out.println("Cálculo del volumen de un cilindro");
        System.out.print("Introduzca el radio en metros: ");
        r = Double.parseDouble(System.console().readLine());
        System.out.print("Introduzca la altura en metros: ");
        h = Double.parseDouble(System.console().readLine());

        System.out.println("El volumen del cilindro es " + matematicas.Geometria.volumenCilindro(r\
, h) + " m3");
    }
}
```

Las líneas

```
import matematicas.Geometria;  
import matematicas.Varias;
```

cargan las clases contenidas en el paquete `matematicas` y, por tanto, todas las funciones contenidas en ellas. No vamos a utilizar el comodín de la forma `import matematicas.*`; ya que está desaconsejado su uso en el estándar de codificación en Java de Google. Observa que se escriben por orden alfabético; como el paquete `matematicas` coincide, se escribe antes `Geometria` y luego `Varias`.

El uso de una función es muy sencillo; hay que indicar el nombre del paquete, el nombre de la clase y finalmente el nombre de la función con los parámetros adecuados. Por ejemplo, como vemos en el programa anterior, `matematicas.Varias.digitos(n)` devuelve el número de dígitos de `n`; siendo `matematicas` el paquete, `Varias` la clase, `digitos` la función y `n` el parámetro que se le pasa a la función.

8.4 Ámbito de las variables

El ámbito de una variable es el espacio donde “existe” esa variable o, dicho de otro modo, el contexto dentro del cual la variable es válida.

Seguramente has utilizado muchas veces variables con nombres como `x`, `i`, `max` o `aux`. El ámbito de cada una de esas variables era el programa en el que estaban declaradas. Fíjate que la `x` del primer programa que aparece en este libro no interfiere para nada con la `x` del siguiente programa que usa una variable con ese mismo nombre. Aunque se llamen igual son variables diferentes.

Cuando se implementan funciones hay que tener muy claro que las variables utilizadas como parámetros (por valor) o las variables que se definen dentro de la función son locales a esa función, es decir, su ámbito es la función y fuera de ella esas variables no existen. Presta atención al programa `Varios.java` y observa que las dos funciones que hay definidas dentro de la clase utilizan sendas variables con nombre `x`. Pues bien, cada una de esas variables son completamente independientes, se trata de dos variables distintas que tienen el mismo nombre pero que son válidas cada una en su propio ámbito.

8.5 Paso de parámetros por valor y por referencia

En Java, como en la mayoría de lenguajes de programación existen dos maneras de pasar parámetros: por valor y por referencia.

Cuando se pasa un parámetro por valor, en realidad se pasa una copia de la variable, únicamente importa el valor. Cualquier modificación que se le haga a la variable que

se pasa como parámetro dentro de la función no tendrá ningún efecto fuera de la misma. Veamos un ejemplo.

```
/**
 * Paso de parámetros por valor
 *
 * @author Luis José Sánchez
 */
public class PruebaParametros1 {
    public static void main(String[] args) {

        int n = 10;

        System.out.println(n);
        calcula(n);
        System.out.println(n);
    }

    public static void calcula(int x) {
        x += 24;
        System.out.println(x);
    }
}
```

La salida del programa anterior es la siguiente:

```
10
34
10
```

A pesar de que la variable que se pasa como parámetro se modifica dentro de la función, los cambios no tienen ningún efecto en el programa principal.

Cuando se pasa un parámetro por referencia, por el contrario, si se modifica su valor dentro de la función, los cambios se mantienen una vez que la función ha terminado de ejecutarse.

En la mayoría de los lenguajes de programación es el programador quien decide cuándo un parámetro se pasa por valor y cuándo se pasa por referencia. En Java no podemos elegir. Todos los parámetros que son de tipo `int`, `double`, `float`, `char` o `String` se pasan siempre por valor mientras que los *arrays* se pasan siempre por referencia. Esto quiere decir que cualquier cambio que efectuemos en un array que se pasa como parámetro permanece cuando termina la ejecución de la función, por lo que hay que tener especial cuidado en estos casos.

```
/**
 * Paso de un array como parámetro
 *
 * @author Luis José Sánchez
 */
public class PruebaParametrosArray {
    public static void main(String[] args) {

        int n[] = {8, 33, 200, 150, 11};
        int m[] = new int[5];

        muestraArray(n);
        incrementa(n);
        muestraArray(n);
    }

    public static void muestraArray(int x[]) {
        for (int i = 0; i < x.length; i++) {
            System.out.print(x[i] + " ");
        }
        System.out.println();
    }

    public static void incrementa(int x[]) {
        for (int i = 0; i < x.length; i++) {
            x[i]++;
        }
    }
}
```

La salida del programa anterior es la siguiente:

```
8 33 200 150 11
9 34 201 151 12
```

Comprobamos, por tanto, que el array que se ha pasado como parámetro se ha modificado en la función y se han conservado los cambios en el programa principal.

Por último, se muestra un programa que contiene funciones que operan con arrays de dos dimensiones.

```
import matematicas.Varias;

/**
 * Paso de un array bidimensional como parámetro
 *
 * @author Luis José Sánchez
 */
public class ArrayBiFunciones {
    public static void main(String[] args) {

        int[][] n = new int[6][9];

        for(int i = 0; i < 6; i++) {
            for(int j = 0; j < 9; j++) {
                n[i][j] = (int)(Math.random()*100000);
            }
        }

        muestraArrayIntBi(n);
    }

    // Funciones //////////////////////////////////////

    /**
     * Devuelve el número de filas de un array bidimensional de
     * números enteros.
     *
     * @param x un array bidimensional de números enteros
     * @return número de filas del array
     */
    public static int filasArrayIntBi(int x[][]) {
        return x.length;
    }

    /**
     * Devuelve el número de columnas de un array bidimensional
     * de números enteros.
     *
     * @param x un array bidimensional de números enteros
     * @return número de columnas del array
     */
    public static int columnasArrayIntBi(int x[][]) {
        return x[0].length;
    }
}
```

```
/**
 * Devuelve el máximo de un array bidimensional
 * de números enteros.
 *
 * @param x un array bidimensional de números enteros
 * @return el valor máximo encontrado en el array
 */
public static int maximoArrayIntBi(int x[][]) {

    int maximo = Integer.MIN_VALUE;

    for (int f = 0; f < filasArrayIntBi(x); f++) {
        for (int c = 0; c < columnasArrayIntBi(x); c++) {
            if (x[f][c] > maximo) {
                maximo = x[f][c];
            }
        }
    }

    return maximo;
}

/**
 * Muestra por pantalla el contenido de un array bidimensional
 * de números enteros.
 *
 * @param x un array bidimensional de números enteros
 */
public static void muestraArrayIntBi(int x[][]) {

    String formatoNumero = "%" + matematicas.Varias.digitos(maximoArrayIntBi(x)) + "d";

    for (int f = 0; f < filasArrayIntBi(x); f++) {
        for (int c = 0; c < columnasArrayIntBi(x); c++) {
            System.out.printf(formatoNumero + " ", x[f][c]);
        }
        System.out.println();
    }
}
```

8.6 Ejercicios



Ejercicios 1-14

Crea una biblioteca de funciones matemáticas que contenga las siguientes funciones. Recuerda que puedes usar unas dentro de otras si es necesario. Observa bien lo que hace cada función ya que, si las implementas en el orden adecuado, te puedes ahorrar mucho trabajo. Por ejemplo, la función `esCapicua` resulta trivial teniendo `voltea` y la función `siguientePrimo` también es muy fácil de implementar teniendo `esPrimo`.

1. **esCapicua**: Devuelve verdadero si el número que se pasa como parámetro es capicúa y falso en caso contrario.
2. **esPrimo**: Devuelve verdadero si el número que se pasa como parámetro es primo y falso en caso contrario.
3. **siguientePrimo**: Devuelve el menor primo que es mayor al número que se pasa como parámetro.
4. **potencia**: Dada una base y un exponente devuelve la potencia.
5. **digitos**: Cuenta el número de dígitos de un número entero.
6. **voltea**: Le da la vuelta a un número.
7. **digitoN**: Devuelve el dígito que está en la posición *n* de un número entero. Se empieza contando por el 0 y de izquierda a derecha.
8. **posicionDeDigito**: Da la posición de la primera ocurrencia de un dígito dentro de un número entero. Si no se encuentra, devuelve -1.
9. **quitaPorDetras**: Le quita a un número *n* dígitos por detrás (por la derecha).
10. **quitaPorDelante**: Le quita a un número *n* dígitos por delante (por la izquierda).
11. **pegaPorDetras**: Añade un dígito a un número por detrás.
12. **pegaPorDelante**: Añade un dígito a un número por delante.
13. **trozoDeNumero**: Toma como parámetros las posiciones inicial y final dentro de un número y devuelve el trozo correspondiente.
14. **juntaNumeros**: Pega dos números para formar uno.



Ejercicio 15

Muestra los números primos que hay entre 1 y 1000.



Ejercicio 16

Muestra los números capicúa que hay entre 1 y 99999.



Ejercicio 17

Escribe un programa que pase de binario a decimal.



Ejercicio 18

Escribe un programa que pase de decimal a binario.



Ejercicio 19

Une y amplía los dos programas anteriores de tal forma que se permita convertir un número entre cualquiera de las siguientes bases: decimal, binario, hexadecimal y octal.



Ejercicios 20-28

Crea una biblioteca de funciones para arrays (de una dimensión) de números enteros que contenga las siguientes funciones:

1. **generaArrayInt**: Genera un array de tamaño n con números aleatorios cuyo intervalo (mínimo y máximo) se indica como parámetro.
2. **mínimoArrayInt**: Devuelve el mínimo del array que se pasa como parámetro.
3. **máximoArrayInt**: Devuelve el máximo del array que se pasa como parámetro.
4. **mediaArrayInt**: Devuelve la media del array que se pasa como parámetro.
5. **estaEnArrayInt**: Dice si un número está o no dentro de un array.
6. **posicionEnArray**: Busca un número en un array y devuelve la posición (el índice) en la que se encuentra.
7. **volteaArrayInt**: Le da la vuelta a un array.
8. **rotaDerechaArrayInt**: Rota n posiciones a la derecha los números de un array.
9. **rotaIzquierdaArrayInt**: Rota n posiciones a la izquierda los números de un array.



Ejercicio 29-34

Crea una biblioteca de funciones para arrays bidimensionales (de dos dimensiones) de números enteros que contenga las siguientes funciones:

1. **generaArrayBiInt**: Genera un array de tamaño $n \times m$ con números aleatorios cuyo intervalo (mínimo y máximo) se indica como parámetro.
2. **filaDeArrayBiInt**: Devuelve la fila i -ésima del array que se pasa como parámetro.
3. **columnaDeArrayBiInt**: Devuelve la columna j -ésima del array que se pasa como parámetro.
4. **coordenadasEnArrayBiInt**: Devuelve la fila y la columna (en un array con dos elementos) de la primera ocurrencia de un número dentro de un array bidimensional. Si el número no se encuentra en el array, la función devuelve el array $\{-1, -1\}$.
5. **esPuntoDeSilla**: Dice si un número es o no punto de silla, es decir, mínimo en su fila y máximo en su columna.
6. **diagonal**: Devuelve un array que contiene una de las diagonales del array bidimensional que se pasa como parámetro. Se pasan como parámetros fila, columna y dirección. La fila y la columna determinan el número que marcará las dos posibles diagonales dentro del array. La dirección es una cadena de caracteres que puede ser "nose" o "neso". La cadena "nose" indica que se elige la diagonal que va del noroeste hacia el sureste, mientras que la cadena "neso" indica que se elige la diagonal que va del noreste hacia el suroeste.



Ejercicio 35

Crea una función con la siguiente cabecera:

```
public static String convierteEnPalotes(int n)
```

Esta función convierte el número n al sistema de palotes y lo devuelve en una cadena de caracteres. Por ejemplo, el 470213 en decimal es el |||| - ||||| | - - || - | - ||| en el sistema de palotes. Utiliza esta función en un programa para comprobar que funciona bien. Desde la función no se debe mostrar nada por pantalla, solo se debe usar `print` desde el programa principal.